# 道路、高架与 Streaming 性能整改设计

## Context

当前 `godot_citys` 的 v2 底盘已经完成了世界尺寸、chunk 生命周期、基础 HLOD、天空与占位城市渲染，但你这轮人工验收暴露出三类不能再靠小修小补绕过去的问题：

1. 道路读起来不像正常城市道路，存在 ribbon 异形边、孤立弧线感和“被侵蚀”的宽度观感。
2. 高架桥仍是视觉占位，不具备桥面厚度、合理净空和可行走/可行驶承托。
3. 高速巡检存在明显卡顿，而当前 streaming 架构仍未把重活真正移出主线程。

本设计的目标不是开启 v3，也不是一次性做完整交通仿真，而是在 **v2 继续收尾** 的前提下，把“城市底盘至少要像个城市”这件事补到可测试、可验证、可继续扩展的程度。

## Alternatives Considered

### Option A: 继续在现有 ribbon 路面上打补丁

做法是继续调 `LOCAL_ROAD_WIDTH_M`、增大近景半径、给桥多加一点高度、在 `Player` 上补一点重力或射线贴地。

优点是快。缺点是根因不会消失：道路仍然不是车道模板，交叉口仍然没有求解，高架仍然不是桥，streaming 仍然会卡。

这个方案不推荐。

### Option B: 保留现有 world/chunk 架构，重做“道路截面 + 交叉口 + 桥面承托 + 分帧 streaming”

宏观 chunk/world 底盘不推翻，但把道路表示升级为“连续道路图 + 车道模板 + 交叉口节点 + 路基/桥面几何”，并把 chunk prepare 变成真正的后台数据准备。

优点是风险可控、改动边界清晰，且能直接解决你本轮提出的核心体验问题。

这是推荐方案。

## Option C: 直接跳到完整 lane graph + 车辆交通 + 立交工程级道路系统

长期看这是对的，但当前阶段过重。它会把问题从“先让道路可信、桥能走、chunk 不卡”扩大成“先做交通仿真平台”，交付速度过慢。

这个方向保留到后续阶段，不作为本轮整改范围。

# Recommended Design

采用 **Option B**，分四个阶段推进。

## Phase A: 贴地移动与桥面承托先成立

先解决“人站不稳、桥走不上去”的底层问题。

实施要求：

- `PlayerController.gd` 增加稳定贴地逻辑：
  - 使用 `floor_snap_length` / `apply_floor_snap()`；
  - 高速巡检模式也必须遵守 floor contact；
  - 在没有地面支撑时才真正进入下落状态。
- 道路和桥面引入独立碰撞层：
  - 普通道路可先使用分段盒碰撞或简化 roadbed collision；
  - 高架桥必须拥有桥面碰撞，不能只依赖下方 terrain。
- 检查 `Player` 的最大可走坡度与桥面法线，保证能连续上桥、下桥、跨 chunk 过渡。

完成标准：

- 手动巡检时，玩家在坡道和桥面上不再表现为“固定 y 值滑行”。
- 玩家可从地面自然驶入或跑上高架桥，并能在桥上稳定移动。

## Phase B: 道路几何从 ribbon 升级为“道路模板系统”

这一步直接解决你截图里的异形路面问题。

设计约束：

- 道路类别固定为四种横断面模板，而不是任意宽灰带：
  - `expressway_elevated`: 双向八车道，含中央分隔与路肩。

    - `arterial`：双向四车道。
    - `local`：双向两车道。
    - `service`：单车道，仅少量出现。
- 每种道路都定义统一的：
    - 车道数；
    - 设计宽度；
    - 中央分隔/路肩/路缘；
    - 默认标线样式；
    - 可接受曲率和坡度。
- 交叉口必须显式建模：
    - 节点类型至少区分 T 字路口、十字路口、斜交路口；
    - 交叉口区域单独求面，不允许两条 ribbon 直接互相穿过去。
- 路基与地形分离：
    - 先确定道路模板和纵断面；
    - 再让地形对道路让形；
    - 不再让道路每个点简单跟地形采样。

结果预期：

- 路面轮廓应保持稳定，远近观察时只增加细节，不改变道路基本外形。
- 道路观感应明显从“多边形步道”转为“市政道路”。

## Phase C: 高架桥升级为可承托的轻量桥体系

桥梁这块不做满工程细节，但必须至少像桥。

设计要求：

- 高架桥由以下元素组成：
    - 桥面 deck；
    - deck 厚度；
    - 支撑柱 / 桥墩；
    - 上下桥过渡段；
    - 桥面碰撞。
- 净空规则：
    - 桥下净空最小设计值以 `5.0m` 为工程下限；
    - 主干高架推荐落在 `6.0m-10.0m` 的常见视觉区间；
    - 不再允许“不到一人高”的伪高架。
- 何时生成桥：

    - 仅在跨越下穿道路、显著地形沟谷或大型节点时生成；
    - 不再使用单纯的随机 marker 抬高。
- 纵断面约束：
    - 进入桥前后必须有合理坡道过渡；
    - 不允许 profile 只在中段突然鼓起。

结果预期：

- 高架桥远看有真实结构感；
- 近看具备桥面厚度与支撑逻辑；
- `Player` 可在桥上稳定巡检。

## Phase D: Streaming 架构去主线程尖峰

这一步解决“高速巡检一卡一卡”的问题。

设计要求：

- 把 chunk lifecycle 拆成真正的三段：
    - `prepare`: 工作线程生成纯数据 payload；
    - `mount`: 主线程按预算挂载已准备好的结果；
    - `retire`: 分帧卸载或回收到对象池。
- `prepare` 阶段至少提前生成：
    - terrain sample grid；
    - road graph slices；
    - 交叉口与 roadbed 数据；
    - 建筑/props placement transforms；
    - bridge deck/support payload。
- `mount` 阶段严格限制预算：
    - 每帧最多挂载有限数量的新 chunk；
    - inspection 模式下对前方 chunk 做优先预取；
    - HUD/debug 更新不得与重建路径耦合。
- `retire` 阶段优先对象池复用 `CityChunkScene` 及其子节点容器，减少 `new()` / `queue_free()` 尖峰。

结果预期：

- 高速巡检时 frame hitch 明显降低；
- 近景 LOD 半径可安全扩大到当前的约两倍而不引入更严重卡顿。

# Data Model Changes

道路数据层新增或收敛为以下字段：

- `road_id`
- `road_class`
- `lane_count_forward`
- `lane_count_backward`
- `section_width_m`
- `median_width_m`
- `shoulder_width_m`
- `grade_profile`
- `elevation_mode`：`ground` / `bridge`
- `bridge_clearance_m`
- `intersection_node_ids`
- `district_id`
- `display_name_id`

这会给你未来说的“主干道、次干道命名并挂指示牌”直接留好口。当前阶段不必立即做真实路牌 mesh，但必须把名字放入数据层。

# 70km 风险门槛

本轮整改必须明确一个现实约束：

- 如果只是验证“底盘逻辑可表达 `70km x 70km` 的城市数据”，当前架构还能继续推进。
- 如果要让玩家真实连续巡检到远离原点数十公里的位置，就必须补一项精度策略：
  - `origin shifting`，或
  - 双精度 large world coordinates 自编译方案。

这不是可选美化项，而是后续真实性与稳定性的前置门槛。

# Verification Plan

自动化验证新增以下重点：

1. `test_player_grounding_and_bridge_snap.gd`
   - 验证玩家在坡道、桥面和桥头连接处保持 floor contact。

2. `test_city_road_section_templates.gd`
    - 验证道路仅生成 `8/4/2/1` 四类模板，宽度不再漂移。
3. `test_city_bridge_deck_collision.gd`
    - 验证桥面存在独立碰撞，玩家可通过。
4. `test_city_intersection_mesh_contract.gd`
    - 验证十字/T 字/斜交路口不会生成自交 ribbon。
5. `test_city_streaming_mount_budget.gd`
    - 验证单帧 mount 数与 retire 数受预算控制。
6. `test_city_origin_distance_warning.gd`
    - 当玩家距离原点超过约定阈值时，输出精度风险提示或触发 origin shift 流程。

人工测试建议：

1. 启动主场景后按 `C` 进入高速巡检模式。
2. 沿主干道持续加速穿越多个 chunk。
3. 检查是否可顺畅上桥、下桥、过交叉口。
4. 观察远中近道路轮廓是否保持同形，只增加细节。
5. 观察高速巡检时是否仍出现明显 hitch。

# Delivery Sequence

建议执行顺序如下：

1. 先做 Phase A，解决贴地与桥面可通行。
2. 再做 Phase B，把 ribbon 路面替换为道路模板系统。
3. 接着做 Phase C，补齐高架桥厚度、净空与承托。
4. 最后做 Phase D，消除高速巡检卡顿。

原因很简单：如果先做性能而不先把道路与桥定义正确，后面仍会在错误几何上优化；那只是把错误做得更快。

# Decision

本设计建议将当前问题定义为 **v2 的真实性与可巡检性整改**，而不是 v3 新范围。后续若进入执行，建议补一份新的 ECN，把“桥梁占位”正式提升为“可承托、可巡检、具备模板化道路截面的轻量桥与道路系统”。